

PyTorch Quick Start

1. Tensors and basic operations

PyTorch uses tensors as its central data structure.

A tensor is similar to a NumPy ndarray, but it can run on a GPU and participate in automatic differentiation.

```
x = torch.tensor([1.0, 2.0, 3.0])
```

```
y = torch.zeros(2, 3)
```

Set `requires_grad=True` when an operation should enter the dynamic graph.

PyTorch Quick Start

2. Automatic differentiation

Calling `backward()` follows the computation graph in reverse.

Gradients accumulate on leaf tensors in their `grad` attributes.

```
x = torch.tensor(2.0, requires_grad=True)
```

```
y = x ** 2
```

```
y.backward()
```

```
print(x.grad) # 4.0
```

Gradients accumulate, so clear them before every training step.

PyTorch Quick Start

3. A minimal training loop

Most training loops repeat five steps:

forward pass, loss, zero gradients, backward pass, parameter update.

```
pred = model(inputs)
```

```
loss = loss_fn(pred, targets)
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

This sequence is the backbone of most PyTorch training code.